COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 11 EXERCISES
STRUCTURED QUERY LANGUAGE (SQL)

BOOK STORE RELATION SCHEMAS

Author(authorId, firstName, lastName)

Book(bookId, title, subject, quantityInStock, price, *authorId*)

BookOrder(orderId, *customerId*, orderYear)

Customer(customerId, firstName, lastName)

OrderDetails(*orderId, bookId*, quantity)

Attribute names in italics are foreign key attributes.

Assumptions

Each author has authored at least one book in the store.

Each book has exactly one Author.

A book may not have sold any copies (i.e., there is no order for the book).

Each order is made by exactly one customer and has one or more associated tuples in OrderDetails (e.g., one order may contain several different books).

REVIEW: CREATE ORDER

Given the foreign keys of the Book Store relations and assuming the referential integrity constraints are included in the create statements, what should be the create order?

Book(bookId, title, subject, quantityInStock, price, *authorId*)

Author(authorId, firstName, lastName)

Customer(customerId, firstName, lastName)

BookOrder(orderId, *customerId*, orderYear)

OrderDetails(*orderId, bookId*, quantity)

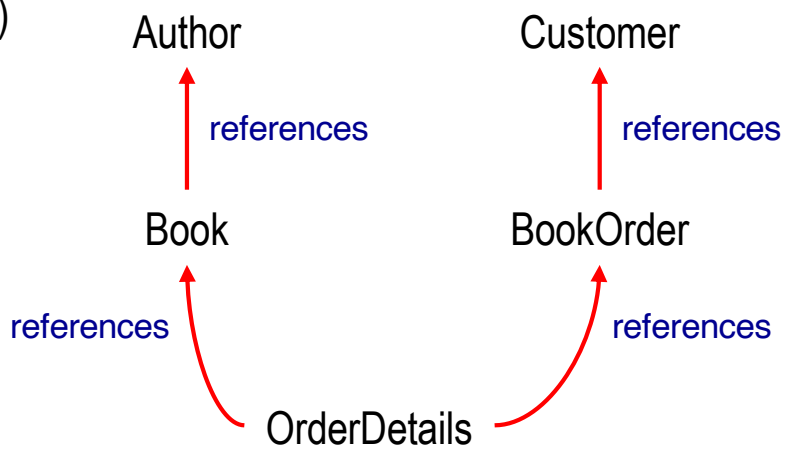

Table	Possible create order					
Author	1	1	2	2	1	3
Customer	2	2	1	1	3	1
Book	3	4	3	4	2	4
BookOrder	4	3	4	3	4	2
OrderDetails	5	5	5	5	5	5

Create Order

Author	*before*	Book
Customer	*before*	BookOrder
Book, BookOrder	*before*	OrderDetails

EXERCISE 1

a) Add an integer attribute numOrders to the Customer relation with values set to 0.
b) Create a view BestCustomers from the Customer relation with attributes customerId, firstName, lastName and numOrders that includes only customers who have placed orders *in every year in which orders have been placed*.

```
alter table Customer add numOrders int default 0;

create view BestCustomers as
with NumOrderYears as
(select customerId, count(distinct orderYear) numYears
  from BookOrder
  group by customerId)
select customerId, firstName, lastName, numOrders
from Customer
where customerId in (select customerId
                     from NumOrderYears
                     where numYears = (select count(distinct orderYear)
                                       from BookOrder));
```

EXERCISE 2

Create a trigger named IncrementOrders that increments the value of the numOrders attribute for a customer in the Customer relation whenever a tuple is inserted into the BookOrder relation for that customer.

```
create or replace trigger IncrementOrders
after insert on BookOrder
for each row
begin
    update Customer
    set numOrders = numOrders + 1
    where customerId = :new.customerId;
end;
```

EXERCISE 3

Find authors who wrote books on both the subjects of Art and Business.
Include in the query result tuples attributes author name (first followed by last).
Order the query result tuples by last name ascending.

Is this condition correct ⟹ **where** subject='Art' **and** subject='Business'? No. Why?

Selects nothing.

Is this condition correct ⟹ **where** subject='Art' **or** subject='Business'? No. Why?

Selects authors who wrote either Art or Business books,
but not necessarily on both subjects.

Authors who wrote books on Art.

Select only those authors in the Art set who are also in the Business set.

Authors who wrote books on Business.

```
select firstName || ' ' || lastName as name
from (select authorId, lastName, firstName
      from Author natural join Book
      where subject='Art'
      intersect
      select authorId, lastName, firstName
      from Author natural join Book
      where subject='Business')
order by lastName;
```

Query result

name
Dan Brown
Pied Piper

Book(bookId, title, subject, quantityInStock, price, *authorId*)

Author(authorId, firstName, lastName)

EXERCISE 3 (CONTD)

Use *only* set membership

Find authors who wrote books on both the subjects of Art and Business. Include in the query result tuples attributes author name (first followed by last). Order the query result tuples by last name ascending.

```
select distinct firstName || ' ' || lastName as name
from Author natural join Book
where subject='Art'
      and authorId in (select authorId
                       from Author natural join Book
                       where subject='Business')
order by name;
```

Query result

name
Dan Brown
Pied Piper

Authors who wrote books on Art (Art set).

Select only those authors in the Art set who are also in the Business set.

Authors who wrote books on Business (Business set).

SQL Note

When specifying **distinct** for a concatenated value in the **select** clause ordering can only be by the concatenated value (i.e., name) and not its parts (i.e., not by lastName or firstName).

Book(bookId, title, subject, quantityInStock, price, *authorId*)

Author(authorId, firstName, lastName)

EXERCISE 4

Find customers who ordered the largest total quantity of books.
Include in the query result tuples attributes customer id, last name and total quantity ordered.
Order the query result tuples by last name ascending.

Is this a correct solution

No! Why?

```
with TotalBooksOrdered as
    (select customerId, lastName, sum(quantity) as totalQuantity
     from Customer natural join BookOrder natural join OrderDetails
     group by customerId, lastName)
select customerId, lastName, max(totalQuantity)
from TotalBooksOrdered
order by lastName;
```

Cannot use an aggregate function in the **select** clause, when retrieving other attributes, without a **group by** clause. Why?

There could be potentially many tuples retrieved, but there is only one aggregate value.

Book(bookId, title, subject, quantityInStock, price, *authorId*)
Author(authorId, firstName, lastName)
Customer(customerId, firstName, lastName)
BookOrder(orderId, *customerId*, orderYear)
OrderDetails(*orderId, bookId*, quantity)

EXERCISE 4 (CONT'D)

Using a subquery with an equality condition.

Find customers who ordered the largest total quantity of books. Include in the query result tuples attributes customer id, last name and total quantity ordered.
Order the query result tuples by last name ascending.

```
select customerId, lastName, sum(quantity) as totalQuantity
from Customer natural join BookOrder natural join OrderDetails
group by customerId, lastName
having sum(quantity)=(select max(sum(quantity))
                      from BookOrder natural join OrderDetails
                      group by customerId)
order by lastName;
```

Select those customers whose total quantity is the largest.

The total quantity of books ordered by each customer.

Query result

customerId	lastName	totalQuantity
4	Chu	711
2	Lam	711
1	Lee	711

EXERCISE 4 (CONT'D)

Using a subquery with >=**all** condition.

Find customers who ordered the largest total quantity of books. Include in the query result tuples attributes customer id, last name and total quantity ordered.
Order the query result tuples by last name ascending.

```
select customerId, lastName, sum(quantity) as totalQuantity
from Customer natural join BookOrder natural join OrderDetails
group by customerId, lastName
having sum(quantity) >=all (select sum(quantity)
                            from BookOrder natural join OrderDetails
                            group by customerId)
order by lastName;
```

Replace = with >=**all**.

Replace **max**(**sum**(quantity)) with **sum**(quantity).

EXERCISE 4 (CONT'D)

Using a derived/ temporary relation.

Find customers who ordered the largest total quantity of books. Include in the query result tuples attributes customer id, last name and total quantity ordered.
Order the query result tuples by last name ascending.

```
with TotalBooksOrdered as
    (select customerId, lastName, sum(quantity) as totalQuantity
     from Customer natural join BookOrder natural join OrderDetails
     group by customerId, lastName)
select customerId, lastName, totalQuantity
from TotalBooksOrdered
where totalQuantity=(select max(totalQuantity)
                     from TotalBooksOrdered)
order by lastName;
```

Select the largest total quantity of books ordered (from the TotalBooksOrdered derived relation).

Calculate the total quantity of books ordered by each customer.

Do not use subqueries.

EXERCISE 5

Do not create any derived/ temporary relations.

Find the authors who wrote books on exactly three different subjects.
Include in the query result tuples attributes author name (first followed by last).
Order the query result tuples by last name ascending.

Is this a correct solution?

No!
Why?

```
select firstName || ' ' || lastName as name,
       count(distinct subject) as numSubjects
from Author natural join Book
group by authorId, firstName, lastName
having numSubjects=3
order by lastName;
```

Retain only those groups having exactly three subjects.

Group by authorId, firstName and lastName.

Join Author and Book on authorId.

Cannot use an alias defined in the **select** clause in the **having** clause
(i.e., cannot use numSubjects in the **having** clause.)

Do not need to return numSubjects in the query result.

Do not use subqueries.

EXERCISE 5 (CONT'D)

Do not create any derived/ temporary relations.

Find the authors who wrote books on exactly three different subjects.
Include in the query result tuples attributes author name (first followed by last).
Order the query result tuples by last name ascending.

Is this a correct solution?

No!
Why?

```
select firstName || ' ' || lastName as name
from Author A
where 3 = (select count(*)
           from Book
           where A.authorId=Book.authorId
           group by A.authorId)
order by lastName;
```

Counts the number of books written by each author and returns true if the author wrote exactly three books.

Selects authors who wrote exactly three books.
(All subject do not have to be different!)
(Pied Piper wrote on only two subjects.)

How to fix this query?

Change **select count**(*) to **select count**(**distinct** subject).
(But should not use subquery!)

Query result

name
Dan Brown
Pied Piper

Do not use subqueries.

EXERCISE 5 (CONT'D)

Do not create any derived/ temporary relations.

Find the authors who wrote books on exactly three different subjects.
Include in the query result tuples attributes author name (first followed by last).
Order the query result tuples by last name ascending.

Is this a correct solution?

Yes, but …

should not use subquery!

```
select firstName || ' ' || lastName as name
from Author
where authorId in (select authorId
                   from Book
                   group by authorId
                   having count(distinct subject)=3
order by lastName;
```

authorId's of authors who wrote books on three different subjects.

Query result

name
Dan Brown

Do not use subqueries.

EXERCISE 5 (CONT'D)

Do not create any derived/ temporary relations.

Find the authors who wrote books on exactly three different subjects.
Include in the query result tuples attributes author name (first followed by last).
Order the query result tuples by last name ascending.

Is this a correct solution?

Yes!

```
select firstName || ' ' || lastName as name
from Author natural join Book
group by authorId, firstName, lastName
having count(distinct subject)=3
order by lastName;
```

Query result

name
Dan Brown

Retain only those groups having exactly three different subjects.

Group by authorId, lastName and firstName.

Join Author and Book on authorId.

Is authorId needed in the **group by** clause?

Yes. Why?

If authorId is not included in the **group by** clause, then the count for two different authors with the same first and last name will be incorrect producing an incorrect result.

Do not use subqueries.

EXERCISE 6

Do not create any derived/ temporary relations.

Find the customers who placed at least three orders in 2025.
Include in the query result tuples attributes last name and number of orders.

Is this a correct solution?

No! Why?

```
select lastName, count(*) as numOrders
from Customer natural join BookOrder
group by customerId, firstName, lastName
having count(*)>3 and orderYear='2025';
```

A *non-aggregated* attribute in the **having** clause must appear in the **group by** clause (i.e., orderYear must appear in the **group by** clause).

How to fix this query?

```
select lastName, count(*) as numOrders
from Customer natural join BookOrder
group by customerId, lastName, orderYear
having count(*)>3 and orderYear='2025';
```

Query result

lastName	numOrders
Lee	4

Do not use subqueries.

EXERCISE 6 (CONT'D)

Do not create any derived/ temporary relations.

Find the customers who placed at least three orders in 2025.
Include in the query result tuples attributes last name and number of orders.

Is this a correct solution?

Yes!

```
select lastName, count(*) as numOrders
from Customer natural join BookOrder
where orderYear='2025'
group by customerId, lastName
having count(*)>3;
```

Select only those groups having more than 3 orders.

Group by customerId and lastName.

Join Customer and BookOrder on customerId and select only orders in 2025.

Are customerId and lastName both needed in the **group by** clause?

Yes. Why?

lastName is needed as it is present in the **select** clause.
customerId is needed to distinguish between customers with the same last name.

EXERCISE 7

Find customers who ordered the first and second highest total number of books. Include in the query result tuples attributes customer name (first followed by last), total number of books ordered, and overall total number of books ordered by all customers.
Order the query result tuples first by total number of books a customer ordered descending and then by last name ascending.

Is this a correct solution?

No!
Why?

```
select firstName || ' ' || lastName as name, sum(quantity) as customerTotal,
       (select (sum(quantity)) from OrderDetails) as overallTotal
from Customer natural join BookOrder natural join OrderDetails
group by customerId, firstName, lastName
order by customerTotal desc
fetch first 2 rows with ties;
```

If there are ties in the first row of the query result, then the customer(s) who have the second highest total number of books ordered will not be returned in the query result.

Only the first two rows will be returned since ties are only included if they occur in the last (i.e., second) row returned.

Also, cannot order the query result tuples attributes by last name.

EXERCISE 7 (CONT'D)

Find customers who ordered the first and second highest total number of books. Include in the query result tuples attributes customer name (first followed by last), total number of books ordered, and overall total number of books ordered by all customers.
Order the query result tuples first by total number of books a customer ordered descending and then by last name ascending.

```
select firstName || ' ' || lastName as name, customerTotal, overallTotal
from (select firstName, lastName, sum(quantity) as customerTotal,
             sum(sum(quantity)) over () as overallTotal
             dense_rank() over (order by sum(quantity) desc) as quantityRank
      from Customer natural join BookOrder natural join OrderDetails
      group by customerId, firstName, lastName)
where quantityRank<=2
order by customerTotal desc, lastName;
```

EXERCISE 7 (CONT'D)

Find customers who ordered the first and second highest total number of books. Include in the query result tuples attributes customer name (first followed by last), total number of books ordered, and overall total number of books ordered by all customers.
Order the query result tuples first by total number of books a customer ordered descending and then by last name ascending.

```
select firstName || ' ' || lastName as name, customerTotal,
        (select sum(quantity) from OrderDetails) as overallTotal
from (select firstName, lastName, sum(quantity) as customerTotal,
                dense_rank() over (order by sum(quantity) desc) as quantityRank
        from Customer natural join BookOrder natural join OrderDetails
        group by customerId, firstName, lastName)
where quantityRank<=2
order by customerTotal desc, lastName;
```

EXERCISE 7 (CONT'D)

Query result

name	customerTotal	overallTotal
Andrew Chu	711	3359
Sam Lam	711	3359
Harry Lee	711	3359
Carl Ip	584	3359
Ron Lee	584	3359